ΛΕΙΤΟΥΡΓΙΚΑ ΣΥΣΤΗΜΑΤΑ

ΑΝΑΦΟΡΑ 2ου ΕΡΓΑΣΤΗΡΙΟΥ

Ομάδα:

Χαράλαμπος Κουνναπής ΑΜ:5401

Στέλιος Σάββα ΑΜ:5404

Παναγιώτης Χριστοδούλου ΑΜ:5501

Γενική ιδέα:

Σε αυτό το εργαστήριο μας ζητείται η υλοποίηση ενός απλού μηχανισμού καταγραφής τροποποιήσεων για το σύστημα αρχείων VFAT εντός της βιβλιοθήκης LKL (Linux Kernel Library). Ο στόχος μας είναι να καταγράφονται σε πραγματικό χρόνο, οι σημαντικές λειτουργίες του συστήματος αρχείων για παράδειγμα δημιουργία αρχείων, καταλογών, και τροποποιήσεις κατά την εκτέλεση εφαρμογών.

Οι εφαρμογές αυτές αποθηκεύονται σε ένα αρχείο του τοπικού συστήματος ώστε να είναι ποιο εύκολα προσβάσιμες και αναγνώσιμες. Η καταγραφή περιλαμβάνει το όνομα της λειτουργίας και βασικές παραμέτρους, με σκοπό την ανάλυση και την επαλήθευση της λειτουργικότητας του VFAT.

Κώδικας cptofs.c

cptofs.c

```
#ifdef __FreeBSD__
#include <sys/param.h>
#endif

#include <sys/param.h>
#include <stdio.h>
#include <time.h>
#include <argp.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#include <libgen.h> // gia to onoma katalogou & vasiko onoma
#include <sys/stat.h>
#include <fcntl.h>
#include <fnmatch.h> // gia antistichisi motivou onomatos
#include <dirent.h>  // gia tis leitourgies eisagogeis
#include <lkl.h>     // pirinas tis vivliothikis Linux Kernel
#include <lkl_host.h> //Leitourgies ipodochis LKL
#include <stdarg.h>  // gia metavlites sinartisis(logging)

// kentriki leitourgia gia log minimaton
static FILE *g_application_log_file = NULL;

/**
 * archikopoiei to sistima katagrafis anoigontas ena archio katagrafis
 * prospathei na anixi prota to /tmp/lkl_cptofs.log kaia stin sinechia to /tmp/lkl_test_alternate.log
 * termatizei se periptosi apotichias anigmatos opoioudipote archiou katagrafis
 */
void log_init(void) {
    g_application_log_file = fopen("/tmp/lkl_cptofs.log", "a");
    if (!g_application_log_file) {
        g_application_log_file = fopen("/tmp/lkl_test_alternate.log", "a");
        if (!g_application_log_file) {
            fprintf(stderr, "Fatal Error: Unable to open or create any log file.\n");
            exit(EXIT_FAILURE);
        } else {
            fprintf(stderr, "Warning: Using alternate log file: /tmp/lkl_test_alternate.log\n");
        }
    } else {
        fprintf(stderr, "Info: Log file opened successfully: /tmp/lkl_cptofs.log\n");
    }
}

/**
 * morfopoiri kai grafei ena minima sto archio katagrafis me chroniko orio(timestamp) kai epipedo(level)
 * level: to epipedo katagrafis (p.x,"INFO", "WARN", "ERROR")
 * format: i simvoloseira morfopoihshs gia to minima
 * args: ta metavlita orismata gia tin simvoloseira morfopoihshs
 */
void log_message(const char *level, const char *format, va_list args) {
    if (!g_application_log_file) return; // Do nothing if log file isn't open

    time_t current_time;
    time(&current_time);
    char time_string_buffer[20];
    strftime(time_string_buffer, sizeof(time_string_buffer), "%Y-%m-%d %H:%M:%S", localtime(&current_time));
```

cptofs.c

```
    // ektiposi chronikou oriou(timestamp) kai epipedou katagrafeis(log level)
    fprintf(g_application_log_file, "[%s] [%s] ", time_string_buffer, level);

    // ektiposi tou pragmatikou onomatos chrisimpoiontas tin morfi kai ta orismata
    va_list args_copy;
    va_copy(args_copy, args); // Copy args because vfprintf consumes it
    vfprintf(g_application_log_file, format, args_copy);
    va_end(args_copy);

    fprintf(g_application_log_file, "\n"); // Add a newline
    fflush(g_application_log_file); // Flush the buffer to ensure immediate write
}

/**
 * katagrafei ena enimerotiko minima
 * format: i simvoloseira morfopoihshs gia to minima
 * Variadic: orismata
 */
void log_info(const char *format, ...) {
    va_list args;
    va_start(args, format);
    log_message("INFO", format, args);
    va_end(args);
}

/**
 * grafei ena proidopoihtiko minima
 */
void log_warn(const char *format, ...) {
    va_list args;
    va_start(args, format);
    log_message("WARN", format, args);
    va_end(args);
}

/**
 * grafei ena proidopoihtiko minima
 */
void log_error(const char *format, ...) {
    va_list args;
    va_start(args, format);
    log_message("ERROR", format, args);
    va_end(args);
}

//simvolosires tekmiriosis gia argp
static const char doc_copy_to_fs[] = "Copy files to a filesystem image (host to LKL FS)";
static const char doc_copy_from_fs[] = "Copy files from a filesystem image (LKL FS to host)";
static const char args_doc_copy_to_fs[] = "-t fstype -i fsimage path... fs_path";
static const char args_doc_copy_from_fs[] = "-t fstype -i fsimage fs_path... path";

// epilogi orismon gia tis grammes entolon argp
static struct argp_option options[] = {
    {"enable-printk", 'p', 0, 0, "show Linux printks"},
    {"partition", 'P', "int", 0, "partition number"},
    {"filesystem-type", 't', "string", 0, "select filesystem type - mandatory"},
```

```
    {"filesystem-image", 'i', "string", 0, "path to the filesystem image - mandatory"},
    {"owner", 'o', "int", 0, "owner of the destination files"},
    {"group", 'g', "int", 0, "group of the destination files"},
    {"selinux", 's', "string", 0, "selinux attributes for destination"},
    {0}, // Sentinel to mark the end of the options array
};

// domi gia tin kratisi enotlon gia to ergalio antigrafis
static struct cl_args {
    int enable_printk;           // gia energopoiisi minimaton sto linux
    int partition_number;        // arithmos gia to mount
    const char *filesystem_type; // tipos archion(p.x, "ext4", "xfs")
    const char *filesystem_image_path; // diadormi tou sistimatos archio pou vriksete i eikona
    int num_paths;               // arithmos diadromon
    char **file_paths;           // pinakas diadromon
    const char *selinux_context; // simvolosira perivallontos asfaleias SELinux
    uid_t owner_uid;             // User ID gia prosopika archia proorismou
    gid_t group_gid;             // Group ID gia omadika archia proorismou
} parsed_command_line_arguments;

static int is_copy_to_filesystem = 0; // Flag: 1 if copying to FS, 0 if copying from FS

/**
 * Parser: sinartisi gai epiloges grammes entolon
 * key: to kleidi epilogis charaktiron
 * arg: ta orismata pou sindeonte me to kleidi epologis
 * state: diktis tis domis argp_state
 * return 0 se periptosi epitichia kai ARGP_ERR_UNKNOWN se periptosi pou to key einai agnosto
 */
static error_t parse_opt(int key, char *arg, struct argp_state *state) {
    struct cl_args *cla = state->input; // Get pointer to our arguments structure

    switch (key) {
        case 'p':
            cla->enable_printk = 1;
            break;
        case 'P':
            cla->partition_number = atoi(arg);
            break;
        case 't':
            cla->filesystem_type = arg;
            break;
        case 'i':
            cla->filesystem_image_path = arg;
            break;
        case 's':
            cla->selinux_context = arg;
            break;
        case 'o':
            cla->owner_uid = atoi(arg);
            break;
        case 'g':
            cla->group_gid = atoi(arg);
            break;
```

```
            break;
        case ARGP_KEY_ARG:
            cla->file_paths = &state->argv[state->next - 1];
            cla->num_paths = state->argc - state->next + 1;
            state->next = state->argc; // Stop further argument parsing
            break;
        default:
            return ARGP_ERR_UNKNOWN;
    }

    return 0;
}

// argp domi gia 'cptofs' (antigrafi apo filesystem)
static struct argp argp_cptofs_config = {
    .options = options,
    .parser = parse_opt,
    .args_doc = args_doc_copy_to_fs,
    .doc = doc_copy_to_fs,
};

// argp domi gia 'cpfromfs' (anigrafi apo filesystem)
static struct argp argp_cpfromfs_config = {
    .options = options,
    .parser = parse_opt,
    .args_doc = args_doc_copy_from_fs,
    .doc = doc_copy_from_fs,
};

static int search_directory_for_files(const char *source_dir_path, const char *destination_dir_path,
                                      const char *filename_match_pattern, uid_t owner_id, gid_t group_id);

/**
 * anigi ena archio pigiaou kofika gai anagnvsi eite ston kentriko ipologisti eite entos tou lkl
 * path: i diadromi pros to archio
 * return File descriptor se periptosi epitichias, se periptosi sfalmatos tipose arnitiki tim
 */
static int open_source_file(const char *path) {
    int file_descriptor;

    if (is_copy_to_filesystem) { // If copying to FS (source is host)
        file_descriptor = open(path, O_RDONLY, 0);
    } else { // If copying from FS (source is LKL)
        file_descriptor = lkl_sys_open(path, LKL_O_RDONLY, 0);
    }

    if (file_descriptor < 0) {
        fprintf(stderr, "Error: Unable to open source file '%s' for reading: %s\n", path,
                is_copy_to_filesystem ? strerror(errno) : lkl_strerror(file_descriptor));
        log_error("Unable to open source file '%s' for reading: %s", path,
                  is_copy_to_filesystem ? strerror(errno) : lkl_strerror(file_descriptor));
    }

    return file_descriptor;
}
```

cptofs.c

```c
/**
 * anigei ena archio me proorismo na grapsei eite ston kentriko ipologisti eite sto sto lkl
 * path: i diadromi tou archiou
 * mode: ta dikaiomata tou archiou
 * owner_id:to UID tou katochou pou tha oristei
 * group_id: to GID tis omadas pou tha oristei
 * File descriptor se periptosi epitchias,ektiposi arnitkis timis se periptosi lathous
 */
static int open_destination_file(const char *path, int mode, uid_t owner_id, gid_t group_id) {
    int file_descriptor;
    int return_code;

    if (is_copy_to_filesystem) {
        file_descriptor = lkl_sys_open(path, LKL_O_RDWR | LKL_O_TRUNC | LKL_O_CREAT, mode);
    } else {
        file_descriptor = open(path, O_RDWR | O_TRUNC | O_CREAT, mode);
    }

    if (file_descriptor < 0) {
        fprintf(stderr, "Error: Unable to open destination file '%s' for writing: %s\n", path,
                is_copy_to_filesystem ? lkl_strerror(file_descriptor) : strerror(errno));
        log_error("Unable to open destination file '%s' for writing: %s", path,
                  is_copy_to_filesystem ? lkl_strerror(file_descriptor) : strerror(errno));
        return file_descriptor; // Return the error code directly
    }

    if (owner_id != (uid_t)-1 && group_id != (gid_t)-1) {
        if (is_copy_to_filesystem) { // Change owner/group in LKL
            return_code = lkl_sys_fchown(file_descriptor, owner_id, group_id);
        } else {
            return_code = fchown(file_descriptor, owner_id, group_id);
        }
        if (return_code) {
            fprintf(stderr, "Error: Unable to set owner/group on '%s': %s\n", path,
                    is_copy_to_filesystem ? lkl_strerror(return_code) : strerror(errno));
            log_error("Unable to set owner/group on '%s': %s", path,
                      is_copy_to_filesystem ? lkl_strerror(return_code) : strerror(errno));
            if (is_copy_to_filesystem) lkl_sys_close(file_descriptor); else close(file_descriptor);
            return -1;
        }
    }
    if (parsed_command_line_arguments.selinux_context && is_copy_to_filesystem) {
        return_code = lkl_sys_fsetxattr(file_descriptor, "security.selinux",
                                        parsed_command_line_arguments.selinux_context,
                                        strlen(parsed_command_line_arguments.selinux_context), 0);
        if (return_code) {
            fprintf(stderr, "Error: Unable to set selinux attribute on '%s': %s\n", path, lkl_strerror(return_code));
            log_error("Unable to set selinux attribute on '%s': %s", path, lkl_strerror(return_code));
        }
    }

    return file_descriptor;
}
```

cptofs.c

```c
/**
 *diavazei dedomena apo ena archio proorismou
 *fd: perigrafei ena archio
 *buffer: to buffer sto opio tha gini i anagnosi
 *length: o megistos arithmos byte pros anagnosi
 *return ton  arithmo byte pou echoun diavastei, tipose minima lathous se error
 */
static int read_from_source(int fd, char *buffer, int length) {
    int bytes_read;

    if (is_copy_to_filesystem) {
        bytes_read = read(fd, buffer, length);
    } else {
        bytes_read = lkl_sys_read(fd, buffer, length);
    }

    if (bytes_read < 0) {
        fprintf(stderr, "Error reading from source file: %s\n",
                is_copy_to_filesystem ? strerror(errno) : lkl_strerror(bytes_read));
        log_error("Error reading from source file: %s",
                  is_copy_to_filesystem ? strerror(errno) : lkl_strerror(bytes_read));
    }
    return bytes_read;
}

/**
 *grafei ta dedomena apo ena archio proorismou
 *fd: perigrafei ena archio
 *buffer: to buffer sto opio tha gini i anagnosi
 *length: o megistos arithmos byte pros anagnosi
 *return ton arithmo byte pou echoun diavastei, tipose minima lathous se error
 */
static int write_to_destination(int fd, char *buffer, int length) {
    int bytes_written;

    if (is_copy_to_filesystem) { // Write to LKL file
        bytes_written = lkl_sys_write(fd, buffer, length);
    } else { // Write to host file
        bytes_written = write(fd, buffer, length);
    }

    if (bytes_written < 0) {
        fprintf(stderr, "Error writing to destination file: %s\n",
                is_copy_to_filesystem ? lkl_strerror(bytes_written) : strerror(errno));
        log_error("Error writing to destination file: %s",
                  is_copy_to_filesystem ? lkl_strerror(bytes_written) : strerror(errno));
    }
    return bytes_written;
}
```

cptofs.c

```c
/**
 *kleini ena perigrafea archiou pigieou kodika
 *fd: o perigrafeas archiou pou tha kleisei
 */
static void close_source_file(int fd) {
    if (is_copy_to_filesystem) {
        close(fd);
    } else {
        lkl_sys_close(fd);
    }
}

/**
 *kleini ena perigrafea archiou pigieou kodika
 *fd: o perigrafeas archiou pou tha kleisei
 */
static void close_destination_file(int fd) {
    if (is_copy_to_filesystem) {
        lkl_sys_close(fd);
    } else {
        close(fd);
    }
}

/**
 *antigrafei to periechomeno enos archiou proelefsis se ena archio proorismou
 * source_path: i diadromi apo to archio proelefsis
 * destination_path: i diadromi tou archiou proorismou
 * file_mode: ta dikaiomata tou neou archiou
 * owner_id: to UID gia to neo archio
 * group_id: to GID gia to neo archio
 * return 0 se periptosi apitichias,arnitiko se periptosi sfalamtos
 */
static int copy_regular_file(const char *source_path, const char *destination_path,
                             int file_mode, uid_t owner_id, gid_t group_id) {
    long bytes_read_count, bytes_to_write, bytes_written_count;
    char data_buffer[4096], *buffer_ptr;
    int overall_return_code = 0;
    int source_fd, destination_fd;

    source_fd = open_source_file(source_path);
    if (source_fd < 0) {
        return source_fd;
    }

    destination_fd = open_destination_file(destination_path, file_mode, owner_id, group_id);
    if (destination_fd < 0) {
        close_source_file(source_fd); // klini to archio porelefsis
        return destination_fd;
    }

    do {
        bytes_read_count = read_from_source(source_fd, data_buffer, sizeof(data_buffer));

        if (bytes_read_count > 0) {
            buffer_ptr = data_buffer;
```

```
        if (bytes_read_count > 0) {
            buffer_ptr = data_buffer;
            bytes_to_write = bytes_read_count;
            do {
                bytes_written_count = write_to_destination(destination_fd, buffer_ptr, bytes_to_write);

                if (bytes_written_count < 0) {
                    overall_return_code = bytes_written_count;
                    goto cleanup_files;
                }

                bytes_to_write -= bytes_written_count;
                buffer_ptr += bytes_written_count;

            } while (bytes_to_write > 0);
        }

        if (bytes_read_count < 0) {
            overall_return_code = bytes_read_count;
        }

    } while (bytes_read_count > 0); // na sinechisi mechri to telos i mechri na vgalei error

cleanup_files:
    close_source_file(source_fd);
    close_destination_file(destination_fd);

    return overall_return_code;
}

/**
 * anakta plirofories statistikon gia mia diadromi proelefsis
 * path: i diadromi pros to archio
 * file_type: tipos archiou gai tin apothikefsi
 * file_mode_out: tipos dikaiomaton tou archiou
 * file_size_out: to megethos tou archiouy
 * modification_time_out: pote tropopoiithike to archio
 * access_time_out: dichni ton chrono pote epitrapike prosvasi sto archio
 * return 0 se periptosi epitichias alios tipose minima lathous
 */
static int stat_source_path(const char *path, unsigned int *file_type_out,
                            unsigned int *file_mode_out, long long *file_size_out,
                            struct lkl_timespec *modification_time_out,
                            struct lkl_timespec *access_time_out) {
    struct stat host_stat;
    struct lkl_stat lkl_stat_info;
    int return_code;

    if (is_copy_to_filesystem) { // pare statistika apo to filesystem
        return_code = lstat(path, &host_stat);
        if (file_type_out)
            *file_type_out = host_stat.st_mode & S_IFMT;
        if (file_mode_out)
            *file_mode_out = host_stat.st_mode & ~S_IFMT;
        if (file_size_out)
            *file_size_out = host_stat.st_size;
```

cptofs.c

```
        if (modification_time_out) {
            modification_time_out->tv_sec = host_stat.st_mtim.tv_sec;
            modification_time_out->tv_nsec = host_stat.st_mtim.tv_nsec;
        }
        if (access_time_out) {
            access_time_out->tv_sec = host_stat.st_atim.tv_sec;
            access_time_out->tv_nsec = host_stat.st_atim.tv_nsec;
        }
    } else { // pare statistika apo to LKL filesystem
        return_code = lkl_sys_lstat(path, &lkl_stat_info);
        if (file_type_out)
            *file_type_out = lkl_stat_info.st_mode & S_IFMT;
        if (file_mode_out)
            *file_mode_out = lkl_stat_info.st_mode & ~S_IFMT;
        if (file_size_out)
            *file_size_out = lkl_stat_info.st_size;
        if (modification_time_out) {
            modification_time_out->tv_sec = lkl_stat_info.lkl_st_mtime;
            modification_time_out->tv_nsec = lkl_stat_info.st_mtime_nsec;
        }
        if (access_time_out) {
            access_time_out->tv_sec = lkl_stat_info.lkl_st_atime;
            access_time_out->tv_nsec = lkl_stat_info.st_atime_nsec;
        }
    }

    if (return_code) {
        fprintf(stderr, "Error: stat_source_path('%s') failed: %s\n",
                path, is_copy_to_filesystem ? strerror(errno) : lkl_strerror(return_code));
        log_error("stat_source_path('%s') failed: %s",
                  path, is_copy_to_filesystem ? strerror(errno) : lkl_strerror(return_code));
    }

    return return_code;
}
```

cptofs.c

```c
/**
 * dimiourgei ena fakelo (dierectory) sto epithimito proorismo eite ston kentriko ipologisti(host) eite ston lkl
 * path: i diadromi pros to archio
 * mode: ta dikeomata tou neou fakelou
 * owner_id: to UID gia to neo archio
 * group_id: to GID gia to neo archio
 * return 0 se periptosi epitichias i ena o katalogos eiparchei idi, tipose minima lathous se periptosi lathous
 */
static int create_destination_directory(const char *path, unsigned int mode, uid_t owner_id, gid_t group_id) {
    int return_code;

    if (is_copy_to_filesystem) { // dimiourgise to directory ston LKL
        return_code = lkl_sys_mkdir(path, mode);
        if (return_code == -LKL_EEXIST) { // If directory already exists, it's not an error
            return_code = 0;
        }
    } else { // dimiourgise to directory ston host
        return_code = mkdir(path, mode);
        if (return_code < 0 && errno == EEXIST) { // If directory already exists, it's not an error
            return_code = 0;
        }
    }
    if (owner_id != (uid_t)-1 || group_id != (gid_t)-1) {
        if (is_copy_to_filesystem) { // allaxe ton/to owner/group ston LKL
            return_code = lkl_sys_chown(path, owner_id, group_id);
        } else { // allaxe ton/to owner/group ston host
            return_code = chown(path, owner_id, group_id);
        }
        if (return_code) {
            fprintf(stderr, "Error: Unable to chown directory '%s': %s\n",
                    path, is_copy_to_filesystem ? strerror(errno) : lkl_strerror(return_code));
            log_error("Unable to chown directory '%s': %s",
                      path, is_copy_to_filesystem ? strerror(errno) : lkl_strerror(return_code));
            return return_code;
        }
    }

    if (return_code) {
        fprintf(stderr, "Error: Unable to create directory '%s': %s\n",
                path, is_copy_to_filesystem ? strerror(errno) : lkl_strerror(return_code));
        log_error("Unable to create directory '%s': %s",
                  path, is_copy_to_filesystem ? strerror(errno) : lkl_strerror(return_code));
    }

    return return_code;
}
```

cptofs.c

```c
/**
 * diavazei ton stocho enos simvolikou sindesmou apo tin archiki topothesia
 * source_symlink_path: i diadromi pros ton simvoliko proorismo
 * output_buffer: endiamesi mnimi gia tin apothikefsi tou stochou sindesis
 * buffer_size: megethos tis endiamesis mnimis exodou
 * return ton arithmo ton byte ston teliko proorismo
 */
static int read_source_symlink(const char *source_symlink_path, char *output_buffer, int buffer_size) {
    int bytes_read;

    if (is_copy_to_filesystem) {
        bytes_read = readlink(source_symlink_path, output_buffer, buffer_size);
    } else {
        bytes_read = lkl_sys_readlink(source_symlink_path, output_buffer, buffer_size);
    }

    if (bytes_read < 0) {
        fprintf(stderr, "Error: Unable to read symlink target for '%s': %s\n", source_symlink_path,
                is_copy_to_filesystem ? strerror(errno) : lkl_strerror(bytes_read));
        log_error("Unable to read symlink target for '%s': %s", source_symlink_path,
                  is_copy_to_filesystem ? strerror(errno) : lkl_strerror(bytes_read));
    }

    return bytes_read;
}

/**
 * dimiorourgei ena simvoliko sindesmo ston proorismo
 * destination_path: i diadromi tou neou simvolikou sindesmou
 * link_target: o telikos stochous tou sindesmou
 * owner_id: to UID gia to neo archio
 * group_id: to GID gia to neo archio
 * return 0 se periptosi epitichias
 */
static int create_destination_symlink(const char *destination_path, const char *link_target,
                                      uid_t owner_id, gid_t group_id) {
    int return_code;

    if (is_copy_to_filesystem) { // dimiourgise ton simvoliko sindesmo ston LKL
        return_code = lkl_sys_symlink(link_target, destination_path);
    } else { // dimiourgise ton simvoliko sindesmo ston host
        return_code = symlink(link_target, destination_path);
    }

    // Set owner and group if specified (using lchown for symlinks on host)
    if (owner_id != (uid_t)-1 || group_id != (gid_t)-1) {
        if (is_copy_to_filesystem) { // Change owner/group in LKL (AT_SYMLINK_NOFOLLOW is important)
            return_code = lkl_sys_fchownat(LKL_AT_FDCWD, destination_path, owner_id, group_id, LKL_AT_SYMLINK_NOFOLLOW);
        } else {
            return_code = lchown(destination_path, owner_id, group_id);
        }
        if (return_code) {
            fprintf(stderr, "Error: Unable to chown symbolic link '%s': %s\n",
                    destination_path, is_copy_to_filesystem ? strerror(errno) : lkl_strerror(return_code));
            log_error("Unable to chown symbolic link '%s': %s",
                      destination_path, is_copy_to_filesystem ? strerror(errno) : lkl_strerror(return_code));
            return return_code;
```

```
    if (return_code) {
        fprintf(stderr, "Error: Unable to create symbolic link '%s' pointing to '%s': %s\n",
                destination_path, link_target, is_copy_to_filesystem ? lkl_strerror(return_code) : strerror(errno));
        log_error("Unable to create symbolic link '%s' pointing to '%s': %s",
                  destination_path, link_target, is_copy_to_filesystem ? lkl_strerror(return_code) : strerror(errno));
    }

    return return_code;
}

/**
 *antigrafei ena simvoliko sindesmo
 *source_symlink_path: i diadromi pros ton simvoliko sindesmo
 * destination_path: i teliki diadromi
 * owner_id: to UID gia to neo archio
 * group_id: to GID gia to neo archio
 * return 0 se periptosi epitichias
 */
static int copy_symbolic_link(const char *source_symlink_path, const char *destination_path,
                              uid_t owner_id, gid_t group_id) {
    int return_code;
    long long symlink_target_size;
    char *target_buffer = NULL;

    // pare to megethos tou symlink target
    return_code = stat_source_path(source_symlink_path, NULL, NULL, &symlink_target_size, NULL, NULL);
    if (return_code) {
        return -1;
    }

    target_buffer = malloc(symlink_target_size + 1); // +1 for null terminator
    if (!target_buffer) {
        fprintf(stderr, "Error: Unable to allocate memory (%lld bytes) for symlink target.\n", symlink_target_size + 1);
        log_error("Unable to allocate memory (%lld bytes) for symlink target.", symlink_target_size + 1);
        return -1;
    }

    // diavase to symlink target
    long long actual_size = read_source_symlink(source_symlink_path, target_buffer, symlink_target_size);
    if (actual_size != symlink_target_size) {
        fprintf(stderr, "Error: read_source_symlink('%s') returned incorrect size: got %lld, expected %lld\n",
                source_symlink_path, actual_size, symlink_target_size);
        log_error("read_source_symlink('%s') returned incorrect size: got %lld, expected %lld",
                  source_symlink_path, actual_size, symlink_target_size);
        return_code = -1;
        goto cleanup_buffer;
    }
    target_buffer[symlink_target_size] = '\0';
    // dimiorgise ena symlink ston proorismo
    return_code = create_destination_symlink(destination_path, target_buffer, owner_id, group_id);
    if (return_code) {
        return_code = -1; // Propagate error
    }
```

```
cleanup_buffer:
    if (target_buffer) {
        free(target_buffer);
    }

    return return_code;
}

/**
 * epexergazete mia memonomeni katachorisi sistimatos archion
 * source_parent_dir: o goneas gia ton katalogo tis katachorisis proelefsis
 * destination_parent_dir: o goneas gia ton katalogo tis katachorisis tou proorismou
 * entry_name: to onoma tis katachorisis
 * owner_id: to UID gia to neo archio
 * group_id: to GID gia to neo archio
 * return 0 se periptosi epitichias
 */
static int process_file_system_entry(const char *source_parent_dir, const char *destination_parent_dir,
                                     const char *entry_name, uid_t owner_id, gid_t group_id) {
    char full_source_path[PATH_MAX];
    char full_destination_path[PATH_MAX];
    struct lkl_timespec access_time, modification_time;
    unsigned int entry_type, entry_mode;
    int return_code;
    snprintf(full_source_path, sizeof(full_source_path), "%s/%s", source_parent_dir, entry_name);
    snprintf(full_destination_path, sizeof(full_destination_path), "%s/%s", destination_parent_dir, entry_name);

    return_code = stat_source_path(full_source_path, &entry_type, &entry_mode, NULL, &modification_time, &access_time);
    if (return_code) {
        // error oti eisai idi sindedemenos ston fakelo
        return return_code;
    }

    // diachirsi diaforon file types
    switch (entry_type) {
        case S_IFREG: // Regular file
            return_code = copy_regular_file(full_source_path, full_destination_path, entry_mode, owner_id, group_id);
            break;
        case S_IFDIR: // Directory
            return_code = create_destination_directory(full_destination_path, entry_mode, owner_id, group_id);
            if (return_code) {
                break; // If directory creation fails, stop
            }
            return_code = search_directory_for_files(full_source_path, full_destination_path, NULL, owner_id, group_id);
            break;
        case S_IFLNK: // Symbolic link
            return_code = copy_symbolic_link(full_source_path, full_destination_path, owner_id, group_id);
            break;
        case S_IFSOCK: // Socket
        case S_IFBLK:  // Block device
        case S_IFCHR:  // Character device
        case S_IFIFO:  // FIFO (named pipe)
        default:
            printf("Info: Skipping '%s': unsupported entry type (0x%x).\n", full_source_path, entry_type);
            log_info("Skipping '%s': unsupported entry type (0x%x).", full_source_path, entry_type);
            break; // No error for unsupported types, just skip
    }
    if (!return_code) {
        if (is_copy_to_filesystem) { // Set timestamps in LKL
            struct lkl_timespec lkl_timestamps[] = { access_time, modification_time };
            return_code = lkl_sys_utimensat(LKL_AT_FDCWD, full_destination_path,
                                            (struct __lkl__kernel_timespec *)lkl_timestamps,
                                            LKL_AT_SYMLINK_NOFOLLOW);
        } else { // Set timestamps on host
            struct timespec host_timestamps[] = {
```

```
        struct timespec host_timestamps[] = {
            { .tv_sec = access_time.tv_sec, .tv_nsec = access_time.tv_nsec, },
            { .tv_sec = modification_time.tv_sec, .tv_nsec = modification_time.tv_nsec, },
        };
        return_code = utimensat(AT_FDCWD, full_destination_path, host_timestamps, AT_SYMLINK_NOFOLLOW);
    }

    if (return_code) {
        fprintf(stderr, "Warning: Failed to set timestamps for '%s': %s\n",
                full_destination_path, is_copy_to_filesystem ? lkl_strerror(return_code) : strerror(errno));
        log_warn("Failed to set timestamps for '%s': %s",
                 full_destination_path, is_copy_to_filesystem ? lkl_strerror(return_code) : strerror(errno));
        return_code = 0; // Treat as non-fatal for overall function return
    }
}

if (return_code) {
    fprintf(stderr, "Error: Failed to process entry '%s', aborting operation.\n", full_source_path);
    log_error("Failed to process entry '%s', aborting operation.", full_source_path);
}

return return_code;
}

/**
 * anigei ena katalogo eite ston ketnriko ipologisti eite entos tou lkl
 * path: i diadromi pros ton katalogo
 * return DIR se periptosi epitichias, null se periptosi lathous
 */
static DIR *open_directory(const char *path) {
    DIR *directory_handle;
    int error_code;

    if (is_copy_to_filesystem) {
        directory_handle = opendir(path);
    } else {
        directory_handle = (DIR *)lkl_opendir(path, &error_code);
    }

    if (!directory_handle) {
        fprintf(stderr, "Error: Unable to open directory '%s': %s\n",
                path, is_copy_to_filesystem ? strerror(errno) : lkl_strerror(error_code));
        log_error("Unable to open directory '%s': %s",
                  path, is_copy_to_filesystem ? strerror(errno) : lkl_strerror(error_code));
    }
    return directory_handle;
}

/**
 * anigei ena fakelo(directory) eite ston host eite ston lkl
 * path: i diadromi pros ton fakelo
 * return DIR se periptosi epitichias, null se periptosi sfalmatos
 */
static const char *read_directory_entry(DIR *dir_handle, const char *path) {
    struct lkl_dir *lkl_dir_handle = (struct lkl_dir *)dir_handle; // Cast for LKL specific functions
    const char *entry_name = NULL;
    const char *error_message = NULL;

    if (is_copy_to_filesystem) {
        struct dirent *directory_entry = readdir(dir_handle);
        if (directory_entry) {
            entry_name = directory_entry->d_name;
        }
    } else {
```

```
    } else {
        struct lkl_linux_dirent64 *lkl_directory_entry = lkl_readdir(lkl_dir_handle);
        if (lkl_directory_entry) {
            entry_name = lkl_directory_entry->d_name;
        }
    }
    if (!entry_name) {
        if (is_copy_to_filesystem) {
            if (errno) {
                error_message = strerror(errno);
            }
        } else {
            if (lkl_errdir(lkl_dir_handle)) {
                error_message = lkl_strerror(lkl_errdir(lkl_dir_handle));
            }
        }
    }

    if (error_message) {
        fprintf(stderr, "Error while reading directory '%s': %s\n", path, error_message);
        log_error("Error while reading directory '%s': %s", path, error_message);
    }
    return entry_name;
}

/**
 * klini to directory handler
 * dir_handle: gia na klisi to directory
 */
static void close_directory_handle(DIR *dir_handle) {
    if (is_copy_to_filesystem) {
        closedir(dir_handle);
    } else {
        lkl_closedir((struct lkl_dir *)dir_handle);
    }
}

/**
 * anazite anadromika ena directory kai antigrafei ta stoichia pou antistichoun metaxei tous
 * source_dir_path: i diadromi apo ton katalogo proelefsis
 * destination_dir_path: i diadromi pros to teliko katalogo
 * filename_match_pattern: motivo gia tin antistichisi ton stichion metaxei tous
 * owner_id: to UID gia to neo archio
 * group_id: to GID gia to neo archio
 * return 0 se periptosi epitichias
 */
static int search_directory_for_files(const char *source_dir_path, const char *destination_dir_path,
                                      const char *filename_match_pattern, uid_t owner_id, gid_t group_id) {
    DIR *directory_handle;
    const char *entry_name;
    int return_code = 0;

    directory_handle = open_directory(source_dir_path);
    if (!directory_handle) {
        return -1; // Error already logged by open_directory
    }
    while ((entry_name = read_directory_entry(directory_handle, source_dir_path))) {
        if (!strcmp(entry_name, ".") || !strcmp(entry_name, "..")) {
            continue;
        }
        if (filename_match_pattern && fnmatch(filename_match_pattern, entry_name, 0) != 0) {
            continue;
        }
        return_code = process_file_system_entry(source_dir_path, destination_dir_path, entry_name, owner_id, group_id);
```

```
        if (return_code) {
            goto cleanup_dir;
        }
    }

cleanup_dir:
    close_directory_handle(directory_handle);

    return return_code;
}

/**
 * elechi an mia diadromi anaferete ston root directory(p.x, ".", "..", "/")
 * afto chrisimopoiite gia na diefkrinisoume an theloume na antigrapsoume ena olokliro directory i ena sigkekrimeno fakelo
 * path: i diadromi tis simvolosiras pou tha elechtei
 * return 1 ena anaferomaste se root alios retun 0
 */
static int is_root_path(const char *path) {
    const char *current_char = path;

    while (*current_char) {
        switch (*current_char) {
            case '.':
                if (current_char > path && current_char[-1] == '.') {
                    return 0;
                }
                break;
            case '/':
                break;
            default:
                return 0;
        }
        current_char++;
    }
    return 1;
}

int copy_single_item(const char *source_item_path, const char *mount_point,
                     const char *destination_path, uid_t owner_id, gid_t group_id) {
    char source_full_path[PATH_MAX];
    char destination_full_path[PATH_MAX];
    char *source_directory_component, *source_base_name_component;

    if (is_copy_to_filesystem) {
        snprintf(source_full_path, sizeof(source_full_path), "%s", source_item_path);
        snprintf(destination_full_path, sizeof(destination_full_path), "%s/%s", mount_point, destination_path);
    } else { // Source is in LKL, destination is on host
        snprintf(source_full_path, sizeof(source_full_path), "%s/%s", mount_point, source_item_path);
        snprintf(destination_full_path, sizeof(destination_full_path), "%s", destination_path);
    }

    if (is_root_path(source_item_path)) {
        log_info("Copying all contents from source directory '%s' to destination '%s'.",
                 source_full_path, destination_full_path);
        return search_directory_for_files(source_full_path, destination_full_path, NULL, owner_id, group_id);
    } else {
        source_directory_component = dirname(strdup(source_full_path));
        source_base_name_component = basename(strdup(source_full_path));

        log_info("Copying single item '%s' from directory '%s' to destination '%s'.",
                 source_base_name_component, source_directory_component, destination_full_path);
```

```
        free(source_directory_component);
        free(source_base_name_component);

        return result;
    }
}

/**
 * Main gia ta cptofs/cpfromfs
 * chirizete tin analisi orismaton, tin archikopoihsh lkl, tis leitourgies tou diskou kai tin antigrafei archion
 * @return 0 ena einai epitichis
 */
int main(int argc, char **argv) {
    log_init();
    log_info("Program started.");

    struct lkl_disk lkl_disk_object;
    long return_code, unmount_return_code;
    int path_index;
    char mount_point_buffer[32];
    unsigned int assigned_disk_id;
    parsed_command_line_arguments.owner_uid = (uid_t)-1;
    parsed_command_line_arguments.group_gid = (gid_t)-1;

    if (strstr(argv[0], "cptofs")) {
        is_copy_to_filesystem = 1;
        return_code = argp_parse(&argp_cptofs_config, argc, argv, 0, 0, &parsed_command_line_arguments);
    } else {
        is_copy_to_filesystem = 0;
        return_code = argp_parse(&argp_cpfromfs_config, argc, argv, 0, 0, &parsed_command_line_arguments);
    }

    if (return_code < 0) {
        log_error("Argument parsing failed.");
        return -1;
    }

    log_info("Arguments parsed successfully.");
    log_info("Filesystem type: %s", parsed_command_line_arguments.filesystem_type);
    log_info("Filesystem image path: %s", parsed_command_line_arguments.filesystem_image_path);
    if (!parsed_command_line_arguments.enable_printk) {
        lkl_host_ops.print = NULL;
    }
    lkl_disk_object.fd = open(parsed_command_line_arguments.filesystem_image_path,
                              is_copy_to_filesystem ? O_RDWR : O_RDONLY);
    if (lkl_disk_object.fd < 0) {
        fprintf(stderr, "Error: Can't open filesystem image '%s': %s\n",
                parsed_command_line_arguments.filesystem_image_path, strerror(errno));
        log_error("Can't open filesystem image '%s': %s",
                  parsed_command_line_arguments.filesystem_image_path, strerror(errno));
        return_code = 1;
        goto cleanup_exit;
    }

    lkl_disk_object.ops = NULL;

    return_code = lkl_init(&lkl_host_ops);
    if (return_code < 0) {
        fprintf(stderr, "Error: LKL initialization failed: %s\n", lkl_strerror(return_code));
        log_error("LKL initialization failed: %s", lkl_strerror(return_code));
        goto cleanup_lkl_context;
    }
    return_code = lkl_disk_add(&lkl_disk_object);
    if (return_code < 0) {
        fprintf(stderr, "Error: Can't add disk to LKL: %s\n", lkl_strerror(return_code));
        log_error("Can't add disk to LKL: %s", lkl_strerror(return_code));
```

```
        fprintf(stderr, "Error: LKL initialization failed: %s\n", lkl_strerror(return_code));
        log_error("LKL initialization failed: %s", lkl_strerror(return_code));
        goto cleanup_lkl_context;
    }
    return_code = lkl_disk_add(&lkl_disk_object);
    if (return_code < 0) {
        fprintf(stderr, "Error: Can't add disk to LKL: %s\n", lkl_strerror(return_code));
        log_error("Can't add disk to LKL: %s", lkl_strerror(return_code));
        goto cleanup_lkl_context;
    }
    assigned_disk_id = return_code;
    return_code = lkl_start_kernel("mem=100M");
    if (return_code < 0) {
        fprintf(stderr, "Error: Failed to start LKL kernel: %s\n", lkl_strerror(return_code));
        log_error("Failed to start LKL kernel: %s", lkl_strerror(return_code));
        goto cleanup_lkl_halt;
    }
    return_code = lkl_mount_dev(assigned_disk_id, parsed_command_line_arguments.partition_number,
                                parsed_command_line_arguments.filesystem_type,
                                is_copy_to_filesystem ? 0 : LKL_MS_RDONLY, // Mount read-only if cpfromfs
                                NULL, mount_point_buffer, sizeof(mount_point_buffer));
    if (return_code) {
        fprintf(stderr, "Error: Can't mount disk: %s\n", lkl_strerror(return_code));
        log_error("Can't mount disk: %s", lkl_strerror(return_code));
        goto cleanup_lkl_halt;
    }
    lkl_sys_umask(0);
    for (path_index = 0; path_index < parsed_command_line_arguments.num_paths - 1; path_index++) {
        return_code = copy_single_item(parsed_command_line_arguments.file_paths[path_index],
                                       mount_point_buffer,
                                       parsed_command_line_arguments.file_paths[parsed_command_line_arguments.num_paths - 1],
                                       parsed_command_line_arguments.owner_uid,
                                       parsed_command_line_arguments.group_gid);
        if (return_code) {
            log_error("Error occurred during file copying.");
            break; // Stop on first copy error
        }
    }
    char device_string[13];
    snprintf(device_string, sizeof(device_string), "/dev/%08x", assigned_disk_id);
    for (;;) {
        return_code = lkl_sys_mount(device_string, mount_point_buffer, (char *)parsed_command_line_arguments.filesystem_type,
                                    LKL_MS_RDONLY | LKL_MS_REMOUNT, NULL);
        if (return_code == 0) {
            break;
        }
        if (return_code == -LKL_EBUSY) { // Device is busy, wait and retry
            struct lkl_timespec sleep_time = {
                .tv_sec = 1,
                .tv_nsec = 0,
            };
            lkl_sys_nanosleep((struct __lkl__kernel_timespec *)&sleep_time, NULL);
            continue;
        } else if (return_code < 0) { // Other error during remount
            fprintf(stderr, "Error: Cannot remount disk read-only: %s\n", lkl_strerror(return_code));
            log_error("Cannot remount disk read-only: %s", lkl_strerror(return_code));
            break; // Exit loop on unrecoverable error
        }
    }
    unmount_return_code = lkl_umount_dev(assigned_disk_id, parsed_command_line_arguments.partition_number, 0, 1000);
    if (return_code == 0) {
        return_code = unmount_return_code;
    if (return_code == 0) {
        return_code = unmount_return_code;
        if (return_code) {
            fprintf(stderr, "Warning: Unmount failed with code: %ld (%s)\n", unmount_return_code, lkl_strerror(unmount_return_code));
            log_warn("Unmount failed with code: %ld (%s)", unmount_return_code, lkl_strerror(unmount_return_code));
        }
    }

cleanup_lkl_halt:
    lkl_sys_halt();
    log_info("Program completed.");

cleanup_lkl_context:
    if (lkl_disk_object.fd >= 0) {
        close(lkl_disk_object.fd);
    }

cleanup_exit:
    if (g_application_log_file) {
        fclose(g_application_log_file);
    }
    return return_code;
}
```

Ο ποιο πάνω κώδικας του cptofs.c είναι οργανωμένος σε μια σειρά συναρτήσεων που είναι υπεύθυνες για τις εργασίες αντιγραφής αρχείων, αλληλεπίδρασης με το σύστημα αρχείων και καταγραφής (logging)

1. **<u>Σύστημα καταγραφής (logging)</u>**
   Το «log_init()» αρχηκοποιει το σύστημα καταγραφής, προσπαθώντας να ανοίξει το αρχείο /tmp/lkl_cptofs.log ή το /tmp/lkl_test_alternate.log σε περίπτωση ένα από τα δυο δεν ανταποκρίνεται.
   Οι συναρτήσεις «log_message(), log_info(), log_warn(), log_error()» είναι υπεύθυνες για την καταγραφή μηνυμάτων με χρονικό όριο και επίπεδο στο αρχείο καταγραφής.
2. **<u>Ανάλυση ορισμάτων γραμμής εντολών (command line argument)</u>**
   Χρησιμοποιεί την βιβλιοθήκη «argp» για την ανάλυση ορισμάτων
   Το «struct cl_args» καθορίζει τη δομή για την αποθήκευση των ορισμάτων όπως ο τύπος συστήματος αρχείων(-t), η διαδρομή εικόνας συστήματος αρχείων(-i), ο αριθμός διαμερίσματος(-p), ο κάτοχος (-o), η ομάδα(-g) και τα χαρακτηριστικά SELinux(-s).
   Η συνάρτηση «parse_opt()» χρησιμοποιείτε για την ανάλυση των επιλογών της γραμμής εντολών.
   Οι δομές «argp_cptofs_config» και « argp_cpfromfs_config» καθορίζουν τις επιλογές και την τεκμηρίωση για τα δυο βοηθητικά βήματα
3. **<u>Αλληλεπίδραση με αρχεία και καταλόγους</u>**
   Οι συναρτήσεις είναι τροποποιημένες έτσι ώστε να μπορούν να λειτουργούν με αρχεία του κεντρικού υπολογιστή αλλά και με αρχεία LKL. Αυτό επιτυγχάνετε μέσω της μεταβλητής «is_copy_to_filesystem» η οποία καθορίζει την διεύθυνση αντιγραφής.
   Οι συναρτήσεις «open_source_file() & open_destination_file()» ανοίγουν αρχεία για ανάγνωση και γραφή αντίστοιχα τόσο στον υπολογιστή όσο και στο LKL. Η «open_destination_file» υποστηρίζει επίσης την ρύθμιση δικαιωμάτων κάτοχου/ομάδας και χαρακτηριστικών SELinux.
   Επίσης οι συναρτήσεις «read_from_source(), write_to_destination()» διαβάζουν και γράφουν δεδομένα από και προς αρχεία με βάση την κατεύθυνση αντιγραφής.
   Οι συναρτήσεις που κλείνουν τους περιγράφεις αρχείων είναι οι «close_source_file(), close_destination_file()»
   Η συνάρτηση «copy_regular_file()» αντιγραφει το περιεχόμενο ενός αρχείου χρησιμοποιώντας ένα buffer 4096 byte για αποτελεσματική αντιγραφή.
   Η «stat_source_path()» ανακτά πληροφορίες στατιστικών. Η «create_destination_directory()» δημιουργεί ένα κατάλογο στον προορισμό

και διαχειρίζεται τους υπάρχοντες καταλόγους και ρυθμίζει τον κάτοχο και ομάδα.
Οι «read_source_symlink(), create_destination_symlink(), copy_symbolic_link()» διαχειρίζονται την ανάγνωση και την δημιουργία συμβολικών συνδέσμων.

4. **<u>Αναδρομική αντιγραφή καταλογών</u>**

   Οι συναρτήσεις «open_directory(), read_directory_entry(), close_directory_handle()» παρέχουν ένα επίπεδο για το άνοιγμα , την ανάγνωση και το κλείσιμο καταλογών, είτε στον κεντρικό υπολογιστή είτε στο lkl

   Το «process_file_system_entry()» επεξεργάζεται μια μεμονωμένη καταχώρηση συστήματος αρχείων καλώντας τις κατάλληλες συναρτήσεις αντιγραφείς.

   Η «search_directory_for_files()» αναζητά αναδρομικά ένα κατάλογο πηγής και αντιγράφει τα στοιχεία που ταιριάζουν μεταξύ τους με ένα προαιρετικό μοτίβο.

5. **<u>Διαχείριση διαδρομής (path management)</u>**
   Το «is_root_path()» ελέγχει εάν μια διαδρομή αναφέρεται σε έναν «root» κατάλογο ώστε να διακρίνει αν πρέπει να αντιγραφεί ολόκληρο το περιεχόμενο ή ένα συγκεκριμένο κομμάτι.
   Η «copy_single_item()» καλείτε όταν πρέπει να αντιγράψει ένα μεμονωμένο στοιχείο.
6. **<u>Βασική λογική (Main)</u>**
   Αρχικοποιεί το σύστημα καταγραφής και αναλύει τα ορίσματα της γραμμής εντολών cptofs. Ακόμη ανοίγει την εικόνα του συστήματος αρχείων και αρχηκοποιει το LKL, προσθέτει τον δίσκο και ξεκινά τον πυρήνα του LKL. Κάνει mount την εικόνα του συστήματος αρχείων στο LKL και προσπαθεί να χρησιμοποιεί τον δίσκο μόνο για ανάγνωση (read-only) , μετα την αντιγραφή. Ακόμη κλείνει τον περιγραφέα αρχείου της εικόνας δίσκου και το αρχείο καταγραφής.

Κώδικας disk.c

disk.c

```c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <time.h>
#include <stdlib.h>
#include <stdint.h>
#include <lkl.h>
#include <lkl_host.h>
#if !defined(__MSYS__) && !defined(__MINGW32__)
#include <sys/stat.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#else
#include <windows.h>
#endif
#include "test.h"
#include "cla.h"

#define LOG_FILE "/tmp/lkl_test.log" // Define the path for the log file

//gennikos perigrafeas archiou gia to log file archikopoiontas to se mi egkiri timi
int g_log_file_descriptor = -1;

/**
 * katagrafei ena minima sto logfile.
 * message: to string message tha graftei sto log file
 */
void log_to_file(const char *message) {
    // mono eggrafei ena to log file anixe epitichos
    if (g_log_file_descriptor != -1) {
        write(g_log_file_descriptor, message, strlen(message));
        write(g_log_file_descriptor, "\n", 1); // prosthese nea grammi
    }
}

// domi gia tin diatirisi parsed command-line arguments eidika gai afti tin dokimi
static struct {
    int enable_printk_output; // Flag to enable Linux printk messages
    const char *disk_image_path; // Path to the disk image file
    const char *filesystem_type; // Type of the filesystem (e.g., "ext4", "xfs")
    int partition_number;        // Partition number to mount
} command_line_arguments;
```

disk.c

```c
//pinakas orismaton
struct cl_arg args[] = {
    {"disk", 'd', "disk file to use", 1, CL_ARG_STR, &command_line_arguments.disk_image_path},
    {"partition", 'P', "partition to mount", 1, CL_ARG_INT, &command_line_arguments.partition_number},
    {"type", 't', "filesystem type", 1, CL_ARG_STR, &command_line_arguments.filesystem_type},
    {0}, // Sentinel to mark the end of the arguments array
};

// katholiki domi diskou lkl
static struct lkl_disk lkl_virtual_disk;
static int lkl_disk_id = -1;

/**
 * prostheti to disk image sto pirina tou lkl
 * return TEST_SUCCESS se periptosi epitichias, TEST_FAILURE se periptosi sfalmatos
 */
int lkl_test_disk_add(void) {
    char log_message_buffer[256];
    snprintf(log_message_buffer, sizeof(log_message_buffer), "Attempting to add disk: %s", command_line_arguments.disk_image_path);
    log_to_file(log_message_buffer);

    // anixe to disk image me vasi to logismiko
#if defined(__MSYS__) || defined(__MINGW32__)
    lkl_virtual_disk.handle = CreateFile(command_line_arguments.disk_image_path, GENERIC_READ | GENERIC_WRITE,
                                         0, NULL, OPEN_EXISTING, 0, NULL);
    if (!lkl_virtual_disk.handle) {
#else
    lkl_virtual_disk.fd = open(command_line_arguments.disk_image_path, O_RDWR);
    if (lkl_virtual_disk.fd < 0) {
#endif
        // an apotichi kane unlink
        goto out_unlink_file;
    }

    lkl_virtual_disk.ops = NULL;

    // vale to disk sto lkl kernel
    lkl_disk_id = lkl_disk_add(&lkl_virtual_disk);
    if (lkl_disk_id < 0) {
        snprintf(log_message_buffer, sizeof(log_message_buffer), "Failed to add disk to LKL: %s", lkl_strerror(lkl_disk_id));
        log_to_file(log_message_buffer);
        goto out_close_handle; // If adding fails, close the file handle
    }
```

disk.c

```
    snprintf(log_message_buffer, sizeof(log_message_buffer), "Disk added successfully with ID: %d", lkl_disk_id);
    log_to_file(log_message_buffer);
    goto out; // Disk added, skip error cleanup

out_close_handle:
// klise to archio
#if defined(__MSYS__) || defined(__MINGW32__)
    CloseHandle(lkl_virtual_disk.handle);
#else
    close(lkl_virtual_disk.fd);
#endif

out_unlink_file:
// unlink/ delete to archio
#if defined(__MSYS__) || defined(__MINGW32__)
    DeleteFile(command_line_arguments.disk_image_path);
#else
    unlink(command_line_arguments.disk_image_path);
#endif

out:
    lkl_test_logf("Disk file descriptor/handle: %x, Disk ID: %d", lkl_virtual_disk.fd, lkl_disk_id);

    if (lkl_disk_id >= 0) {
        return TEST_SUCCESS;
    }

    log_to_file("Failed to add disk due to an earlier error.");
    return TEST_FAILURE;
}

/**
 * afairei to disk apo ton pirina tou lkl kai kleieni to file handle
 * return TEST_SUCCESS an einai epitichis, TEST_FAILURE an echi sfalma
 */
int lkl_test_disk_remove(void) {
    char log_message_buffer[256];
    snprintf(log_message_buffer, sizeof(log_message_buffer), "Attempting to remove disk with ID: %d", lkl_disk_id);
    log_to_file(log_message_buffer);

    // svise to disk apo to lkl kernel
    int return_code = lkl_disk_remove(lkl_virtual_disk);
```

disk.c

```c
// klise to disk image file handle
#if defined(__MSYS__) || defined(__MINGW32__)
    CloseHandle(lkl_virtual_disk.handle);
#else
    close(lkl_virtual_disk.fd);
#endif

    if (return_code == 0) {
        log_to_file("Disk removed successfully.");
        return TEST_SUCCESS;
    }

    snprintf(log_message_buffer, sizeof(log_message_buffer), "Failed to remove disk: %s", lkl_strerror(return_code));
    log_to_file(log_message_buffer);
    return TEST_FAILURE;
}

static char mount_point_buffer[32];
LKL_TEST_CALL(mount_device, lkl_mount_dev, 0, lkl_disk_id, command_line_arguments.partition_number,
              command_line_arguments.filesystem_type, 0, NULL, mount_point_buffer, sizeof(mount_point_buffer))

/**
 * aposindei tin porigoumeni siskevi apo ton pirina tou lkl
 * return TEST_SUCCESS an einai epitichis, TEST_FAILURE an echi sfalma
 */
static int lkl_test_umount_dev(void) {
    char log_message_buffer[256]; // Declare buffer here
    log_to_file("Attempting to unmount device...");

    // allazei to directory to opio vriskese mesa sto lkl se root gia na pofigi "device busy" errors
    long chdir_result = lkl_sys_chdir("/");
    // Unmount the device
    long umount_result = lkl_umount_dev(lkl_disk_id, command_line_arguments.partition_number, 0, 1000);

    if (chdir_result == 0 && umount_result == 0) {
        log_to_file("Device unmounted successfully.");
        return TEST_SUCCESS;
    } else {
        snprintf(log_message_buffer, sizeof(log_message_buffer),
                 "Failed to unmount device. Chdir result: %ld, Umount result: %ld (%s)",
                 chdir_result, umount_result, lkl_strerror(umount_result));
        log_to_file(log_message_buffer);
        return TEST_FAILURE;
    }
}

struct lkl_dir *g_directory_handle;
```

disk.c

```c
/**
 * anigi to directory mesa sto LKL filesystem.
 * return TEST_SUCCESS an einai epitichis, TEST_FAILURE an parousisasti sfalma
 */
static int lkl_test_opendir(void) {
    int error_code;
    char log_message_buffer[256]; //dilonoume ena buffer
    snprintf(log_message_buffer, sizeof(log_message_buffer), "Opening directory: %s", mount_point_buffer);
    log_to_file(log_message_buffer);

    // anigei to directory chrisimopiontas tin entoli tou LKL's opendir
    g_directory_handle = lkl_opendir(mount_point_buffer, &error_code);
    lkl_test_logf("lkl_opendir(%s) = %d %s\n", mount_point_buffer, error_code,
                  lkl_strerror(error_code));

    if (error_code == 0) {
        log_to_file("Directory opened successfully.");
        return TEST_SUCCESS;
    }

    snprintf(log_message_buffer, sizeof(log_message_buffer), "Failed to open directory: %s", lkl_strerror(error_code));
    log_to_file(log_message_buffer);
    return TEST_FAILURE;
}

/**
 * diavazei ta entries apo to opened directory.
 * TEST_SUCCESS an einai epitichis, TEST_FAILURE an parousiasti sfalma
 */
static int lkl_test_readdir(void) {
    struct lkl_linux_dirent64 *directory_entry;
    int characters_written = 0;
    char log_message_buffer[256]; // dilonoume ena buffer

    log_to_file("Reading directory entries...");

    // kanei Loop anamesa sta directory entries mechri na epistrafei NULL
    while ((directory_entry = lkl_readdir(g_directory_handle))) {
        //  dilosi sto directory to entry name
        characters_written += lkl_test_logf("%s ", directory_entry->d_name);
        snprintf(log_message_buffer, sizeof(log_message_buffer), "Read directory entry: %s", directory_entry->d_name);
        log_to_file(log_message_buffer);

        //prosthetei nea grammi an afti pou einai gini poli megali
        if (characters_written >= 70) {
            lkl_test_logf("\n");
            characters_written = 0;
```

disk.c

```c
            // break gia na apofigi tin sichisi an ta entries einai poli megala
            break;
        }
    }

    // elechos gai sfalmata mesa sto directory entries
    if (lkl_errdir(g_directory_handle) == 0) {
        log_to_file("Directory read successfully.");
        return TEST_SUCCESS;
    }

    snprintf(log_message_buffer, sizeof(log_message_buffer), "Failed to read directory: %s", lkl_strerror(lkl_errdir(g_directory_handle)));
    log_to_file(log_message_buffer);
    return TEST_FAILURE;
}

LKL_TEST_CALL(closedir_handle, lkl_closedir, 0, g_directory_handle);
LKL_TEST_CALL(change_to_mount_point, lkl_sys_chdir, 0, mount_point_buffer);
LKL_TEST_CALL(start_lkl_kernel, lkl_start_kernel, 0, "mem=128M loglevel=8");
LKL_TEST_CALL(stop_lkl_kernel, lkl_sys_halt, 0);

// pinakas ton tests pou tha ektelestoun apo to LKL test framework
struct lkl_test tests[] = {
    LKL_TEST(disk_add),
    LKL_TEST(start_lkl_kernel),
    LKL_TEST(mount_device),
    LKL_TEST(change_to_mount_point),
    LKL_TEST(opendir),
    LKL_TEST(readdir),
    LKL_TEST(closedir_handle),
    LKL_TEST(umount_dev),
    LKL_TEST(stop_lkl_kernel),
    LKL_TEST(disk_remove),
};
/**
 * sinartisi main tou disk lkl
 * return 0 se periptosi epitchias, return -1 se periptosi sfalmatos
 */
int main(int argc, const char **argv) {
    int test_run_result;
    // anigma tou log file gia eggrafi
    g_log_file_descriptor = open(LOG_FILE, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (g_log_file_descriptor < 0) {
        perror("Error: Failed to open log file");
        return -1;
    }

    if (parse_args(argc, argv, args) < 0) {
        log_to_file("Error: Failed to parse command-line arguments.");
        close(g_log_file_descriptor);
        return -1;
    }
    if (!command_line_arguments.disk_image_path || !command_line_arguments.filesystem_type) {
        fprintf(stderr, "Error: 'disk' and 'type' parameters are required.\n");
        log_to_file("Error: Missing required 'disk' or 'type' command-line parameters.");
        close(g_log_file_descriptor);
        return -1;
    }
    // prosarmozoume to LKL host print function sto prosarmosmeno logger
    lkl_host_ops.print = lkl_test_log;
    //archikopoiisi tou LKL perivallon
    lkl_init(&lkl_host_ops);
    // trexe ta apetoumena tests
    test_run_result = lkl_test_run(tests, sizeof(tests) / sizeof(struct lkl_test),
                                   "disk %s", command_line_arguments.filesystem_type);
    // katharise to lkl perivallon
    lkl_cleanup();
    // klise to log file
    close(g_log_file_descriptor)
    return test_run_result;
}
```

Ο ποιο πάνω κώδικας του disk.c λειτουργεί ως ένα βοηθητικό πρόγραμμα δοκιμών για την αλληλεπίδραση με εικόνες δίσκου χρησιμοποιώντας το Linux Kernel Library (LKL). Επιδεικνύει βασικές λειτουργίες όπως η προσθήκη και η αφαίρεση δίσκων, η προσάρτηση (mount) &(unmount) συστημάτων αρχείων ,την αλλαγή καταλογών και την ανάγνωση περιεχομένων καταλογών εντός του περιβάλλοντος LKL.

1. **<u>Σύστημα καταγραφής (logging system)</u>**
   Το «LOG_FILE» ορίζεται ως «/tmp/lkl_test.log» για την καταγραφή μηνυμάτων. Ακόμη το «g_log_file_descriptor» είναι ένας καθολικός περιγραφεας αρχείου για το αρχείο καταγραφής.
   Η «log_to_file(const char *message)» συνάρτηση γραφεί ένα μήνυμα στο αρχείο καταγραφής προσθέτοντας μια νέα γραμμή.
   Η κυριά συνάρτηση (main) ανοίγει το αρχείο καταγραφής με δικαιώματα εγγραφής, δημιουργίας και αποκοπής.
2. **<u>Ανάλυση ορισμάτων γραμμής εντολών(command line arguments)</u>**
   Το «command_line_arguments» είναι μια στατική δομή που αποθηκεύει τα αναλυμένα ορίσματα μαζί με του «enable_printk_output» (για ενεργοποίηση μηνυμάτων printk) , «disk_image_path»(διαδρομή εικόνας δίσκου), «filesystem_type»(ο τύπος αρχείου) και «partition_number».
   Τέλος η συνάρτηση main καλεί το «parse_args» για να αναλύσει τα ορίσματα και να ελέγξει επίσης εάν οι παράμετροι disk & type είναι υποχρεωτικές.
3. **<u>Διαχείριση δίσκου LKL(LKL disk management)</u>**
   Η συνάρτηση «lkl_virtual_disk» είναι μια στατική δομη lkl_disk που αναπαριστά τον εικονικό δίσκο. Στην συνέχεια η «lkl_disk_id» είναι μια ακέραια μεταβλητή για την αποθήκευση του δίσκου από το LKL. Ακόμη η «lkl_test_disk_add()» προσπαθεί να ανοίξει την εικόνα δίσκου(Disk image) σε κατάσταση ανάγνωσης/γραφής. Προσθέτει τον δίσκο στον πυρήνα του συστήματος χρησιμοποιώντας το «lkl_disk_add()» και χειρίζεται επίσης τον καθαρισμό αρχείου σε περίπτωση σφάλματος. Τέλος η «lkl_test_disk_remove()» αφαιρεί τον δίσκο από τον πυρήνα του lkl χρησιμοποιώντας «lkl_disk_remove()» και κλείνει τον περιγραφεα αρχείου δίσκου.

4. **<u>Λειτουργίες Συστήματος Αρχείων LKL(LKL Filesystem Operations)</u>**
   Το «mount_point_buffer» είναι ένα buffer για την αποθήκευση του σημείου προσάρτησης. Ακόμη η «mount_device» καλεί το «lkl_mount_dev» για να προσαρτήσει την συσκευή στο LKL με βάσει το αναγνωριστικό δίσκου, τον αριθμό και τον τύπο αρχείου. Στην συνέχεια η «lkl_test_umount_dev()» συνοδεύετε από την κλήση «lkl_sys_chdir("/")» οπού μετακινείτε στο κατάλογο «root» για να αποφευχθούν σφάλματα. Επίσης το «g_directory_handle» είναι ένας καθολικός δείκτης για την διαχείριση καταλόγου, η συνάρτηση «lkl_test_opendir()» ανοίγει ένα κατάλογο εντός του LKL χρησιμοποιώντας το «lkl_opendir()". Η «lkl_test_readdir()» διαβάζει καταχωρίσεις από τον ανοιχτό κατάλογο του LKL.

5. **<u>Διαχείριση πυρήνα LKL(LKL Kernel Management)</u>**
   Χρησιμοποίει την «start_lkl_kernel» για να ξεκινήσει τον πυρήνα του LKL με καθορισμένα ορίσματα. Στην συνέχεια χρησιμοποιεί «stop_lkl_kernel» για να σταματήσει τον πυρήνα του lkl χρησιμοποιώντας «lkl_sys_halt()» και αυτά τα δυο γίνονται μέσω του «LKL_TEST_CALL».
6. **<u>Πλαίσιο δοκιμών(Testing Framework)</u>**
   Χρησιμοποιεί το «tests[]» οπού είναι ένας πίνακας «lkl_test» που περιέχει τη σειρά των δοκιμών προς εκτέλεση. Για παράδειγμα προσθήκη δίσκου, εκκίνηση του πυρήνα και συνεχίζετε.
   Τέλος στην “main()” συνάρτηση αρχικοποιει το περιβάλλον LKL με «lkl_init()», εκτελεί τα καθορισμένα test χρησιμοποιώντας «lkl_test_run()» και καθαρίζει ο περιβάλλον LKL με «lkl_cleanup()» και κλείνει το αρχείο γραφείς.

Έξοδοί εκτελέσεις :

1. Αφού εκτελέσουμε make -j8 για την μεταγλώττιση του κώδικα εκτελούμε «make run-tests ./tests/run.py» στον φάκελο «tools/lkl» και περνούμε τα ποιο κάτω αποτελέσματα

```
myy601@myy601lab2:~/lkl-source/tools/lkl$ make run-tests ./tests/run.py
./tests/run.py
  SUITE                 boot
  TEST        ok
  TEST        ok        mutex
  TEST        ok        semaphore
  TEST        ok        join
  TEST        ok        start_kernel
  TEST        ok        getpid
  TEST        ok        syscall_latency
  TEST        ok        umask
  TEST        ok        umask2
  TEST        ok        creat
  TEST        ok        close
  TEST        ok        failopen
  TEST        ok        open
  TEST        ok        write
  TEST        ok        lseek_cur
  TEST        ok        lseek_end
  TEST        ok        lseek_set
  TEST        ok        read
  TEST        ok        fstat
  TEST        ok        mkdir
  TEST        ok        stat
  TEST        ok        nanosleep
  TEST        ok        pipe2
  TEST        ok        epoll
  TEST        ok        mount_fs_proc
  TEST        ok        chdir_proc
  TEST        ok        open_cwd
  TEST        ok        getdents64
  TEST        ok        close_dir_fd
  TEST        ok        chdir_root
  TEST        ok        umount_fs_proc
  TEST        ok        lo_ifup
  TEST        ok        gettid
  TEST        ok        syscall_thread
  TEST        ok        many_syscall_threads
  TEST        ok        stop_kernel
  SUITE                 disk_vfat
  TEST        ok        prepfs_vfat
  TEST        ok        disk_add
  TEST        ok        start_lkl_kernel
  TEST        ok        mount_device
  TEST        ok        change_to_mount_point
  TEST        ok        opendir
  TEST        ok        readdir
  TEST        ok        closedir_handle
  TEST        ok        umount_dev
  TEST        ok        stop_lkl_kernel
  TEST        ok        disk_remove
Summary: 2 suites run, 47 tests, 47 ok, 0 not ok, 0 skipped
make: Nothing to be done for 'tests/run.py'.
```

2. Στην συνέχεια εκτελούμε την εντολή «cat /tmp/lkl_tests.log»

```
myy601@myy601lab2:~/lkl-source/tools/lkl$ cat /tmp/lkl_test.log
Attempting to add disk: /tmp/disk
Disk added successfully with ID: 0
Opening directory: /mnt/0000fe00
Directory opened successfully.
Reading directory entries...
Read directory entry: .
Read directory entry: ..
Directory read successfully.
Attempting to unmount device...
Device unmounted successfully.
Attempting to remove disk with ID: 0
Disk removed successfully.
```

3. Στην συνέχεια εκτελούμε στον φάκελο «tools/lkl/tests» την εντολή «./disk.sh -t vfat» που δημιουργεί το αρχείο δίσκου και το κάνει mount πριν εκτελέσει τις διάφορες λειτουργίες. Ακόμα εκτελούμε τις εντολές «./cptofs -p -I vfat lklfuse.o / » και «cat /tmp/lkl_cptofs.log» στο φάκελο tools/lkl για να κάνει log το αρχείο και να σιγουρευτούμε ότι ο κώδικας μας λειτουργεί σωστά.

```
myy601@myy601lab2:~/lkl-source/tools/lkl/tests$ cd ..
myy601@myy601lab2:~/lkl-source/tools/lkl$ ./cptofs -p -i /tmp/disk -t vfat lklfuse.o /
Info: Log file opened successfully: /tmp/lkl_cptofs.log
[    0.000000] Linux version 6.1.0 (myy601@myy601lab2) (gcc (Debian 12.2.0-14) 12.2.0, GNU ld (GNU Binutils for Debian) 2.40) #15 Mon May 26 06:58:35 EEST 2
[    0.000000] memblock address range: 0x7f7de1c00000 - 0x7f7de8000000
[    0.000000] Zone ranges:
[    0.000000]   Normal   [mem 0x00007f7de1c00000-0x00007f7de7ffffff]
[    0.000000] Movable zone start for each node
[    0.000000] Early memory node ranges
[    0.000000]   node   0: [mem 0x00007f7de1c00000-0x00007f7de7ffffff]
[    0.000000] Initmem setup node 0 [mem 0x00007f7de1c00000-0x00007f7de7ffffff]
[    0.000000] Built 1 zonelists, mobility grouping on.  Total pages: 25250
[    0.000000] Kernel command line:  mem=100M virtio_mmio.device=328@0x1000000:1
[    0.000000] Dentry cache hash table entries: 16384 (order: 5, 131072 bytes, linear)
[    0.000000] Inode-cache hash table entries: 8192 (order: 4, 65536 bytes, linear)
[    0.000000] mem auto-init: stack:all(zero), heap alloc:off, heap free:off
[    0.000000] Memory: 100768K/102400K available (3454K kernel code, 488K rwdata, 773K rodata, 101K init, 313K bss, 1632K reserved, 0K cma-reserved)
[    0.000000] SLUB: HWalign=32, Order=0-3, MinObjects=0, CPUs=1, Nodes=1
[    0.000000] NR_IRQS: 4096
[    0.000000] lkl: irqs initialized
[    0.000000] clocksource: lkl: mask: 0xffffffffffffffff max_cycles: 0x1cd42e4dffb, max_idle_ns: 881590591483 ns
[    0.000000] lkl: time and timers initialized (irq2)
[    0.000899] pid_max: default: 4096 minimum: 301
[    0.003388] Mount-cache hash table entries: 512 (order: 0, 4096 bytes, linear)
[    0.003391] Mountpoint-cache hash table entries: 512 (order: 0, 4096 bytes, linear)
[    0.008623] printk: console [lkl_console0] enabled
[    0.008646] clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 19112604462750000 ns
[    0.009696] NET: Registered PF_NETLINK/PF_ROUTE protocol family
[    0.009761] lkl_pci: probe of lkl_pci failed with error -1
[    0.016754] vgaarb: loaded
[    0.016830] clocksource: Switched to clocksource lkl
[    0.016940] NET: Registered PF_INET protocol family
[    0.018743] IP idents hash table entries: 2048 (order: 2, 16384 bytes, linear)
[    0.019552] tcp_listen_portaddr_hash hash table entries: 512 (order: 0, 4096 bytes, linear)
[    0.019563] Table-perturb hash table entries: 65536 (order: 6, 262144 bytes, linear)
[    0.019567] TCP established hash table entries: 1024 (order: 1, 8192 bytes, linear)
[    0.019573] TCP bind hash table entries: 1024 (order: 2, 16384 bytes, linear)
[    0.019586] TCP: Hash tables configured (established 1024 bind 1024)
[    0.019602] UDP hash table entries: 128 (order: 0, 4096 bytes, linear)
[    0.019615] UDP-Lite hash table entries: 128 (order: 0, 4096 bytes, linear)
[    0.019658] PCI: CLS 0 bytes, default 32
[    0.019691] virtio-mmio: Registering device virtio-mmio.0 at 0x1000000-0x1000147, IRQ 1.
[    0.027746] workingset: timestamp_bits=62 max_order=15 bucket_order=0
[    0.028997] io scheduler mq-deadline registered
[    0.029012] io scheduler kyber registered
[    0.030485] virtio_blk virtio0: 1/0/0 default/read/poll queues
[    0.030817] virtio_blk virtio0: [vda] 614400 512-byte logical blocks (315 MB/300 MiB)
[    0.031721]  vda:
[    0.032222] NET: Registered PF_INET6 protocol family
[    0.034118] Segment Routing with IPv6
[    0.034140] In-situ OAM (IOAM) with IPv6
[    0.034152] sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver
[    0.043436] Warning: unable to open an initial console.
```

```
myy601@myy601lab2:~/lkl-source/tools/lkl$ cat /tmp/lkl_cptofs.log
[2025-05-26 03:22:05] [INFO] Program started.
[2025-05-26 03:22:05] [INFO] Arguments parsed successfully.
[2025-05-26 03:22:05] [INFO] Filesystem type: vfst
[2025-05-26 03:22:05] [INFO] Filesystem image path: /tmp/disk
[2025-05-26 03:22:05] [ERROR] Can't mount disk: No such device
[2025-05-26 03:22:05] [INFO] Program completed.
[2025-05-26 03:32:16] [INFO] Program started.
[2025-05-26 03:32:16] [INFO] Arguments parsed successfully.
[2025-05-26 03:32:16] [INFO] Filesystem type: vfat
[2025-05-26 03:32:16] [INFO] Filesystem image path: /tmp/disk
[2025-05-26 03:32:16] [INFO] Copying single item 'lklfuse.0' from directory '.' to destination '/mnt/0000fe00//'.
[2025-05-26 10:50:41] [INFO] Program started.
[2025-05-26 10:50:41] [INFO] Arguments parsed successfully.
[2025-05-26 10:50:41] [INFO] Filesystem type: vfat
[2025-05-26 10:50:41] [INFO] Filesystem image path: /tmp/disk
[2025-05-26 10:50:41] [INFO] Copying single item 'lklfuse.o' from directory '.' to destination '/mnt/0000fe00//'.
```